

Git Guide

A Friendly Introduction for Beginners

Tailored for VS Code & Google Colab

Beginner Friendly

VS Code

Google Colab

GitHub

This guide takes you from zero Git knowledge to confidently tracking changes, collaborating on GitHub, and using Git every day in VS Code and Google Colab.

COVERS

Concepts & Setup

Branching

Git in VS Code

Daily Workflow

GitHub Remotes

Git in Colab

Fixing Mistakes

Pull Requests

Cheat Sheet

Table of Contents

1. What is Git and Why Should I Care?
2. Core Concepts: The Mental Model
3. Installing Git
4. First-Time Setup
5. Your First Repository
6. The Daily Git Workflow
7. Branching – Working in Parallel
8. Working with GitHub (Remote Repos)
9. Git Inside VS Code
10. Git with Google Colab
11. Fixing Common Mistakes
12. Collaboration & Pull Requests
13. Quick-Reference Cheat Sheet

What is Git and Why Should I Care?

Git is a **version control system** – think of it as an unlimited Undo button for your entire project, combined with a time machine and a collaboration tool. Instead of files named *report_final_v3_ACTUALLY_FINAL.ipynb*, Git tracks every change and lets you travel back to any point in history.

[i] **Git** $\neq$ **GitHub**. Git is the tool that runs on your computer. GitHub is a website that stores your Git history in the cloud so teammates can access it.

Why bother?

- **Safety net** – roll back any bad change instantly.
- **History** – see exactly what changed, when, and why.
- **Teamwork** – work in parallel without overwriting each other.
- **Industry standard** – every software company in the world uses it.

Core Concepts: The Mental Model

Before touching the terminal, it helps to understand the four places your code can live in Git's world:

Zone	What it is	Analogy
Working Directory	Files on your disk right now	<i>Your desk</i>
Staging Area	Changes marked "ready to save"	<i>Tray on your desk</i>
Local Repository	Full history saved on this computer	<i>Personal filing cabinet</i>
Remote Repository	Copy on GitHub, shared with the team	<i>Shared office archive</i>

Key vocabulary

- **Repository (repo)** – the project folder that Git tracks.
- **Commit** – a snapshot of your project at a point in time. Like Save + a sticky note.
- **Branch** – an independent line of development. Experiment safely without touching main code.
- **Merge** – combining two branches back together.
- **Clone** – downloading a remote repo to your computer.
- **Push** – uploading your commits to GitHub.
- **Pull** – downloading the latest commits from GitHub.

Installing Git

Windows

1. Go to <https://git-scm.com/download/win> and download the installer.
2. Run it – leave all defaults and click Next until Finish.
3. Open **Git Bash** and verify:

> Verify installation

```
git --version
```

You should see something like *git version 2.44.0.windows.1*.

macOS

Open **Terminal** and run `git -version`. If not installed, macOS will prompt you to install Xcode Command Line Tools – click Install.

Linux (Ubuntu / Debian)

```
sudo apt update  
sudo apt install git
```

[Tip] Google Colab: Git is already installed – skip this section entirely when working in a notebook.

First-Time Setup

Git needs to know who you are so it can label your commits. Run these two commands once:

> Set your identity

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

[i] Use the same email you registered with on GitHub – this links your commits to your profile.

Optional but recommended

Set VS Code as the editor Git opens when it needs your input:

> Set VS Code as Git editor

```
git config --global core.editor "code --wait"
```

Verify all your settings:

```
git config --list
```

Your First Repository

Option A – Start from scratch

> Create a new local repo

```
mkdir my_project      # create a new folder
cd my_project         # move into it
git init              # turn it into a Git repo
```

Git creates a hidden `.git` folder inside – that is where all the history lives. **Never delete it.**

Option B – Clone an existing GitHub repo (most common)

> Download from GitHub

```
git clone https://github.com/username/repo-name.git
cd repo-name
```

[Tip] VS Code shortcut: Press **Ctrl+Shift+P** (or **Cmd+Shift+P** on Mac), type *Git: Clone*, and paste your URL. No terminal needed!

The Daily Git Workflow

Most of your day-to-day work follows the same three-step loop:



Check what has changed

> See the state of your repo

```
git status
```

Red = changed but not staged. **Green** = staged, ready to commit.

Stage your changes

```
git add filename.py    # stage one specific file
git add .              # stage ALL changed files
```

Commit – save the snapshot

> Write a short, descriptive message

```
git commit -m "Add data cleaning function"
```

[Tip] Write messages in the present tense, keep them short.

Good: *“Fix bug in preprocessing”* **Bad:** *“stuff”* or *“asdfgh”*

View your history

```
git log                # full log
git log --oneline      # compact one-line-per-commit view
```

Branching – Working in Parallel

A branch is like a parallel copy of your project where you can try new ideas without affecting the main code. The default branch is called **main**.

> Create, work on, and merge a branch

```
git branch                # list all branches
git checkout -b feature/my-feature # create + switch in one
                                command

# Do your work, stage and commit as usual ...

git checkout main          # switch back to main
git merge feature/my-feature # bring in the changes
```

[!] Always commit or stash your work before switching branches – Git will warn you if you have unsaved changes.

Delete a finished branch

```
git branch -d feature/my-feature # safe delete (only if already
                                merged)
```

Working with GitHub (Remote Repos)

Push your commits to GitHub

> Upload your work

```
git push origin main
```

origin is the nickname Git gives to the GitHub URL. **main** is the branch name.

Pull the latest changes from GitHub

Before starting work each day, pull to get your teammate's latest commits:

> Download teammate's latest work

```
git pull origin main
```

Link a local repo to GitHub for the first time

Create an empty repo on GitHub (no README), then:

> First push to GitHub

```
git remote add origin https://github.com/username/repo-name.git
git branch -M main
git push -u origin main
```

[Tip] The **-u** flag sets the upstream link – after this first push you can simply type `git push` without specifying `origin main` every time.

Git Inside VS Code

VS Code has full built-in Git support – you rarely need the terminal for everyday tasks.

The Source Control panel

Click the **branch icon** in the left sidebar (or press **Ctrl+Shift+G**). You will see:

- **Changes** – modified files (same as `git status`).
- **Staged Changes** – files you have clicked + on.
- A **message box** at the top for your commit message.
- A **Commit** button to save all staged changes.

Quick reference – VS Code Git actions

Action	What it does
Click + on a file	Stage that file (<code>git add</code>)
Click + on Changes	Stage all files (<code>git add .</code>)
Type message, Commit	Commit staged changes
... menu > Push	Push to GitHub (<code>git push</code>)
... menu > Pull	Pull from GitHub (<code>git pull</code>)
Branch name (status bar)	Switch or create branches

[Tip] Install the **GitLens** extension from the VS Code marketplace for powerful history browsing and line-by-line blame views.

Resolving merge conflicts

When two people edit the same line, VS Code highlights the conflict with inline buttons: **Accept Current Change**, **Accept Incoming Change**, or **Accept Both**. Click your choice, save, then stage and commit.

Git with Google Colab

Colab runs on temporary cloud machines that reset each session. Git is pre-installed, but you need to re-clone and re-configure each time you start.

Clone your repo at the start of each session

> Run in a Colab code cell

```
!git clone https://github.com/username/repo-name.git
%cd repo-name
```

Set your identity in Colab

```
!git config --global user.name "Your Name"
!git config --global user.email "you@example.com"
```

Authenticate with a Personal Access Token (PAT)

Colab cannot use a saved password. You need a GitHub Personal Access Token:

1. On GitHub: **Settings** → **Developer settings** → **Personal access tokens** → **Generate new token**.
2. Give it the **repo** scope and copy it (starts with *ghp_...*).

> Push using your token

```
!git push https://YOUR_TOKEN@github.com/username/repo-name.git main
```

[!] Never hard-code your token in a notebook you commit! Use Colab's **Secrets** panel (key icon in the left sidebar) to store it safely.

Save notebook changes back to GitHub

```
!git add my_notebook.ipynb
!git commit -m "Update analysis"
!git push origin main
```

Fixing Common Mistakes

Undo the last commit but keep the changes

```
git reset --soft HEAD~1
```

Your files stay as-is; only the commit is removed. Re-stage and re-commit freely.

Discard all uncommitted changes to one file

```
git checkout -- filename.py
```

[!] This permanently discards unsaved changes. Use with caution!

Fix the last commit message

```
git commit --amend -m "Corrected commit message"
```

Only do this *before* you push to GitHub.

Un-stage a file

```
git restore --staged filename.py
```

See exactly what changed

```
git diff filename.py          # unstaged changes
git diff --staged filename.py  # staged changes
```

Temporarily shelve work in progress

```
git stash          # shelve current changes
git stash pop      # restore shelved changes
```

Collaboration & Pull Requests

A **Pull Request (PR)** is GitHub's way of saying: *"I have changes on my branch – please review and merge them into main."*

The collaboration workflow

Step 1 – Create a branch for your feature:

```
git checkout -b feature/my-feature
```

Step 2 – Work, stage, and commit as normal.

Step 3 – Push the branch to GitHub:

```
git push origin feature/my-feature
```

Step 4 – On GitHub.com a banner appears: *"Compare & pull request"* – click it.

Step 5 – Write a description and click **Create Pull Request**.

Step 6 – Your teammate reviews, approves, and merges.

[Tip] Always pull from main before creating a new branch – this minimises merge conflicts.

.gitignore – files Git should never track

Create a file named **.gitignore** in the root of your project:

```
> .gitignore example
```

```
__pycache__/  
*.pyc  
.env  
data/raw/  
.ipynb_checkpoints/  
*.csv
```

Quick-Reference Cheat Sheet

Command	What it does
<code>git init</code>	Initialise a new local repository
<code>git clone <url></code>	Download a remote repo to your machine
<code>git status</code>	Show changed / staged files
<code>git add <file></code>	Stage a specific file
<code>git add .</code>	Stage all changes
<code>git commit -m "msg"</code>	Save a snapshot with a message
<code>git log --oneline</code>	Compact commit history
<code>git diff</code>	Show unstaged changes
<code>git push origin main</code>	Upload commits to GitHub
<code>git pull origin main</code>	Download latest commits from GitHub
<code>git branch</code>	List all branches
<code>git checkout -b <name></code>	Create and switch to a new branch
<code>git checkout main</code>	Switch to the main branch
<code>git merge <branch></code>	Merge a branch into current branch
<code>git branch -d <name></code>	Delete a merged branch
<code>git reset --soft HEAD~1</code>	Undo last commit, keep changes staged
<code>git restore --staged <file></code>	Un-stage a file
<code>git checkout -- <file></code>	Discard all changes to a file (careful!)
<code>git stash</code>	Shelve uncommitted changes temporarily
<code>git stash pop</code>	Restore shelved changes

[Tip] Bookmark this page! These 20 commands cover 99 % of everyday Git tasks.

*Happy coding! Every expert was once a beginner –
after a week of daily use, Git becomes second nature.*